2 If there is back edge, there will be cycle and if there is no back edge, it means there are tree edges only.

We can just run DFS. → Running time depends on # edges.

Check every single edge one by one until max |V| edges and return the point that have cycle.

```
Check    TEdges (ParentsNode, PresentNode, VisitedNode, adjMatrix){
  For each Node a in V {
    For Each Edge b of Node a{
      if no mark on b,
      then  Mark b and TEdges = TEdges + 1
      if TEdges is same = |V|
        return "It has Cycle"
    }
  }
  return no Cycle
}
```

This algorithm runs at O(V) time. because if it has a back edge, it means that it will be found before |V| since it is in acyclic forest that is $|E| \leq |V| - 1$

2/

Algorithm

For equality constraint $C_i = C_j$ create the graph and do DFS to get every single components

Use for each node x_i.

If it is connected in a^{th},

Then assign and mark it.

For Inequality constraint $x_i \neq x_j$.

If x_i and x_j are in different connect component,

Then constraints are unsatisfiable

else

constraints are satisfiable.

Graph is made for DFS to find every components and equality constraint. It assign each node again for a_{th} component. The points happen in different connected components for Inequality constraint.

Changed Algorithm to make it efficient

make a graph by using adj list & n(nodes)

Add edge to equality of x_i & x_j

Use DFS and Int counter = 0

If DFS visit node, → set value of node to counter value.

If DFS → Inequality constraint, then counter = counter + 1

See Inequalities values.

If Inequality and counter value are same → return null

If equality constraints are same, → return satisfiable.

equality constraint make equivalence class. If m constraints are taken over n variables the condition of inequality won't satisfiable the algorithm. If every constraints satisfy, there won't inequality constraints. If every constraints are satisfiable, → condition of equality is satisfiable.

$x_2 \neq x_1$ → constraints are unsatisfiable. Thus, algorithm correct for not satisfiable constraints.

3.

(a) v → edges to nodes $w_1 \sim w_k$

Recursive formula for cost value ⇒ $\min\{p_v, \min_{(v,w)\in E} cost[w]\}$

This formula is easy for directed acyclic graph.

For each node $v \in V$, Sort DAG topologically

↳ $cost[v] = \min\{p_v, \min_{(v,w)\in E} cost[w]\}$ ⇒ This algorithm is efficient and linear to fill out

(b) Let two nodes u & v are connected component

↳ u that is reachable is equal to v. ⇒ $cost[u] = cost[v]$

In same strong connection, Don't need to differentiate nodes

Algorithm

Get component G that is strongly connected

For each strongly connected component → let price P_c as component C. The smallest price of every nodes → $P_c = \min_{v \in C} p_w$

Run DAG and return component metacosts → $cost^*[C]$

Lastly, ⇒ for strong same connection, → each node $v \in C$ ⇒ $cost[v] = cost^*[C]$

The extended algorithm takes linear time since it's composed of strong components.

4

(a) It is an MST still because the change only increases some spanning trees cost and T cost didn't change.

(b) Include e in tree and create a cycle. Remove the heaviest edge in cycle that is found in linear time to get new tree T. T include a smallest edge across all cut of G. Thus, T is MST.

(c) There is no need to be changed because the tree is still MST when edge weight in tree decrease.

(d) Get rid of e from Tree to get two components. Contain lightest edge across the cut to have new T. Build T by using the cut and tell T is MST.

5. Each of flight create edge and airport creates node in the graph. Flight time match to edge weight. Difference between previous flight's arrive time and future flight's departing time need to be greater than (c(a)) This condition will reduce # edges for traverse through graph to have best solution. We can use DFT for this problem. Once we get paths that goes to destination airport and report the cost about path, the traversal ends. If node has visited already, the cycle needs to be able to follow and removed. The running time for DFS is $O(m+n)$

nodes

flights

6

(a) G → Graph w → Cost s → Source

(b) DAG Algorithm ⇒ Sort graph vertices ⇒ Declar (G, S) ⇒

for vertex, V → Sorted order ⇒ V ∈ G

Time complexity → $O(V+E) + O(E) + O(V) = O(V+E)$

vertices edge

(c) $\boxed{O(m \log n)}$

for int i = 1 to n